

Discovering Modelfactors

#pragma omp parallel

#pragma omp single

3. #pragma omp taskloop

```
None
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp taskloop
        for (...) {
            // trabajo paralelo
        }
    }
}
```

Flujo:

1. parallel: se crean, por ejemplo, 4 hilos (H0, H1, H2, H3).
2. single: solo H0 entra en el bloque.
3. taskloop: H0 divide el bucle en, digamos, 20 tareas de 5 iteraciones cada una.
4. Esas 20 tareas se ponen en la cola.
5. H0, H1, H2, H3 empiezan a coger tareas de la cola hasta completar las 100 iteraciones.

Every 2.0s: squeue

```

JOBID PARTITION    NAME      USER ST       TIME  NODES NODELIST(REASON)
315635 execution submit-s paco1115 R        0:35      1 boada-18
315634 execution submit-s paco1211 R        0:18      1 boada-19
315600 interacti _interac paco1115 R       38:00      1 boada-11
315624 interacti _interac paco1110 R        5:56      1 boada-11
315620 interacti _interac paco1110 R        7:25      1 boada-11
315582 interacti _interac paco1104 R       1:02:41      1 boada-11
315614 interacti _interac paco1109 R        17:43      1 boada-11
315625 interacti _interac paco1109 R         5:06      1 boada-12
315584 interacti _interac paco1108 R       1:00:49      1 boada-11
315585 interacti _interac paco1106 R        59:41      1 boada-11
315575 interacti _interac paco1102 R       1:44:02      1 boada-11
315615 interacti _interac paco1111 R        17:28      1 boada-11
315604 interacti _interac paco1105 R        31:28      1 boada-11
315581 interacti _interac paco1211 R       1:03:23      1 boada-11
315617 interacti _interac paco1105 R        14:58      1 boada-11
315590 interacti _interac paco1113 R        53:01      1 boada-11
315619 interacti _interac paco1211 R         7:30      1 boada-11
315627 interacti _interac cpds1112 R         3:28      1 boada-12
315611 interacti _interac paco1114 R        21:30      1 boada-11
315599 interacti _interac paco1114 R        38:06      1 boada-11
315579 interacti _interac paco1107 R       1:06:47      1 boada-11
315621 interacti _interac paco1107 R         7:20      1 boada-11
315623 interacti _interac      eva R         6:02      1 boada-11
315577 interacti _interac      eva R       1:15:26      1 boada-11

```

Overview of whole program execution metrics						
Number of processors	1	2	4	8	16	32
Elapsed time (sec)	1.22	0.91	0.74	0.76	1.14	2.04
Speedup	1.00	1.35	1.66	1.61	1.07	0.60
Efficiency	1.00	0.67	0.41	0.20	0.07	0.02

Table 1: Analysis done on Wed Sep 24 09:43:34 AM CEST 2025, paco1114

Conclusiones inmediatas

1. **Buena hasta 4 hilos** ($S_4 \approx 1.66$) y **degradación clara** a partir de 8: $T_8 > T_4$, $T_{\{16\}}$ y $T_{\{32\}}$ empeoran → **sobrecoste de paralelización y/o granularidad demasiado fina** (muchas tareas pequeñas, mucha sincronización) y/o **desbalance**. Esto es exactamente lo que el laboratorio espera que identifiques en la versión inicial.

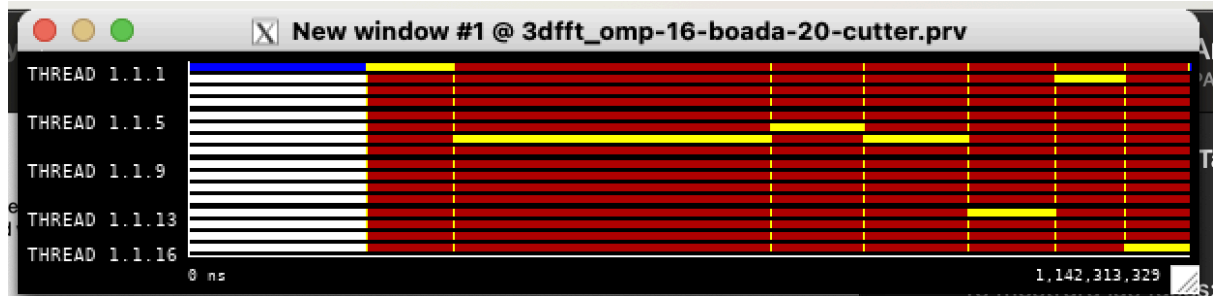
Overview of the Efficiency metrics in parallel fraction, $\phi=83.14\%$						
Number of processors	1	2	4	8	16	32
Global efficiency	99.12%	71.18%	46.84%	22.59%	6.69%	1.71%
Parallelization strategy efficiency	99.12%	93.97%	90.59%	88.08%	63.84%	39.39%
Load balancing	100.00%	99.47%	98.02%	97.81%	97.78%	96.76%
In execution efficiency	99.12%	94.47%	92.41%	90.05%	65.28%	40.71%
Scalability for computation tasks	100.00%	75.75%	51.71%	25.65%	10.48%	4.34%
IPC scalability	100.00%	79.93%	58.87%	37.13%	22.61%	35.36%
Instruction scalability	100.00%	99.43%	98.31%	96.17%	92.10%	87.67%
Frequency scalability	100.00%	95.31%	89.35%	71.83%	50.32%	13.99%

Table 2: Analysis done on Wed Sep 24 09:43:34 AM CEST 2025, paco1114

Statistics about explicit tasks in parallel fraction						
Number of processors	1	2	4	8	16	32
Number of explicit tasks executed (total)	17920.0	35840.0	71680.0	143360.0	286720.0	458752.0
LB (number of explicit tasks executed)	1.0	0.95	0.85	0.78	0.66	0.79
LB (time executing explicit tasks)	1.0	1.0	0.99	0.98	0.98	0.98
Time per explicit task (average us)	56.08	37.02	27.12	27.36	33.49	50.53
Overhead per explicit task (synch %)	0.14	5.49	9.04	11.72	52.22	148.93
Overhead per explicit task (sched %)	0.74	0.9	1.29	1.71	4.3	5.01
Number of taskwait/taskgroup (total)	1792.0	1792.0	1792.0	1792.0	1792.0	1792.0

Table 3: Analysis done on Wed Sep 24 09:43:34 AM CEST 2025, paco1114

1.2. Paraver



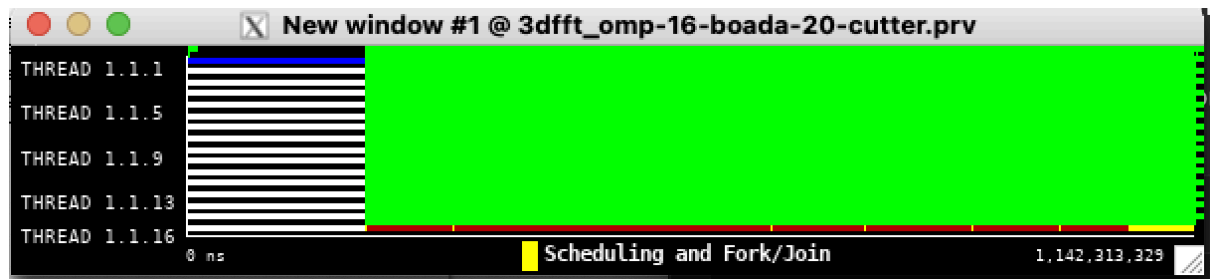
1.2.3 Paraver profiles

2D thread state				
	Running	Not created	Synchronization	Scheduling and Fork/Join
THREAD 1.1.1	69.96 %	-	21.51 %	8.52 %
THREAD 1.1.2	51.67 %	17.70 %	27.25 %	3.39 %
THREAD 1.1.3	53.42 %	17.70 %	28.88 %	0.00 %
THREAD 1.1.4	53.49 %	17.70 %	28.81 %	0.00 %
THREAD 1.1.5	53.44 %	17.70 %	28.86 %	0.00 %
THREAD 1.1.6	52.31 %	17.70 %	20.86 %	9.13 %
THREAD 1.1.7	48.79 %	17.70 %	29.16 %	4.35 %
THREAD 1.1.8	53.32 %	17.70 %	28.98 %	0.00 %
THREAD 1.1.9	53.27 %	17.70 %	29.03 %	0.00 %
THREAD 1.1.10	53.47 %	17.70 %	28.83 %	0.00 %
THREAD 1.1.11	53.33 %	17.70 %	28.97 %	0.00 %
THREAD 1.1.12	53.22 %	17.70 %	29.08 %	0.00 %
THREAD 1.1.13	52.18 %	17.70 %	21.60 %	8.53 %
THREAD 1.1.14	53.29 %	17.70 %	29.01 %	0.00 %
THREAD 1.1.15	53.21 %	17.70 %	29.10 %	0.00 %
THREAD 1.1.16	50.90 %	17.70 %	29.11 %	2.30 %
Total	859.27 %	265.46 %	439.04 %	36.23 %
Average	53.70 %	17.70 %	27.44 %	2.26 %
Maximum	69.96 %	17.70 %	29.16 %	9.13 %
Minimum	48.79 %	17.70 %	20.86 %	0.00 %
StDev	4.37 %	0.00 %	2.97 %	3.38 %
Avg/Max	0.77	1.00	0.94	0.25

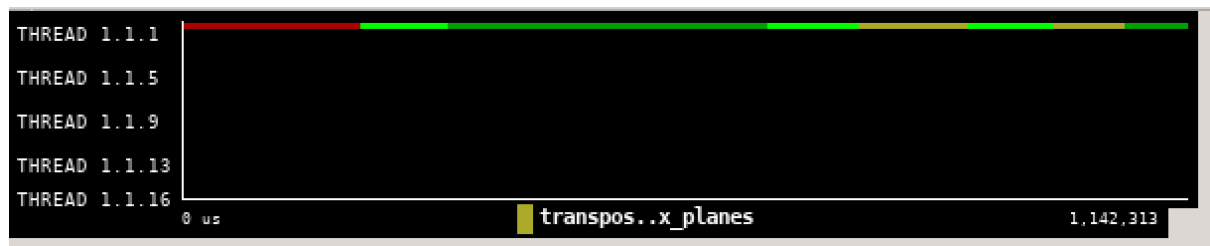
- Los hilos **solo trabajan útilmente ~50 % del tiempo**, el resto se pierde en **sincronización (27 %)** y **costes de scheduling (2–9 %)**.
- Esto **corroborar lo que decían las tablas de Modelfactors**: la escalabilidad está limitada no por falta de trabajo paralelo ($\phi=83\%$ es alto), sino por **overheads de gestión de tareas y sincronización excesiva**.
- El **desbalance no es el problema** (todos los hilos tienen porcentajes muy parecidos).

1.2.4. Paraver Flags

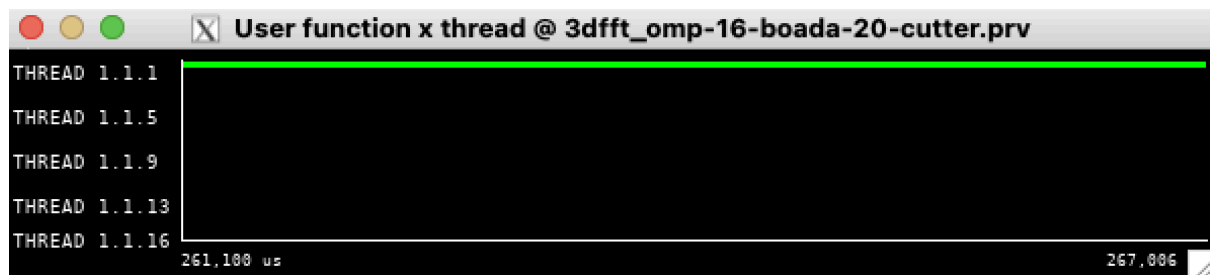
Aparecen apelotonados



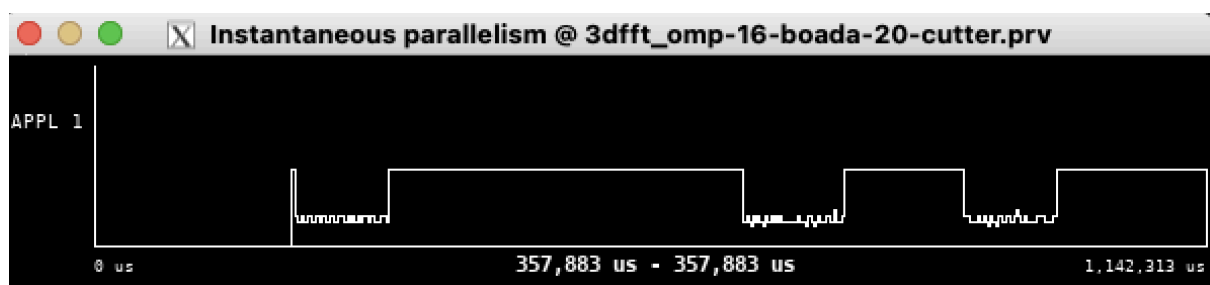
1.2.5. Example 3dfft



Únicamente crea las tareas el thread 1 y los demás consumen de esas tareas.



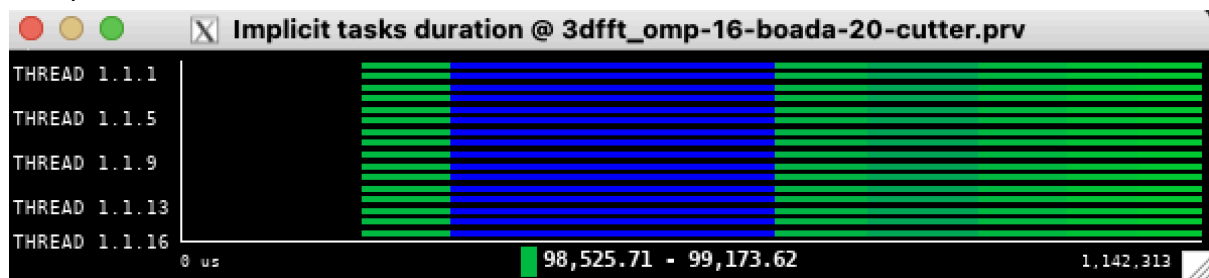
Puntos 2 y 3



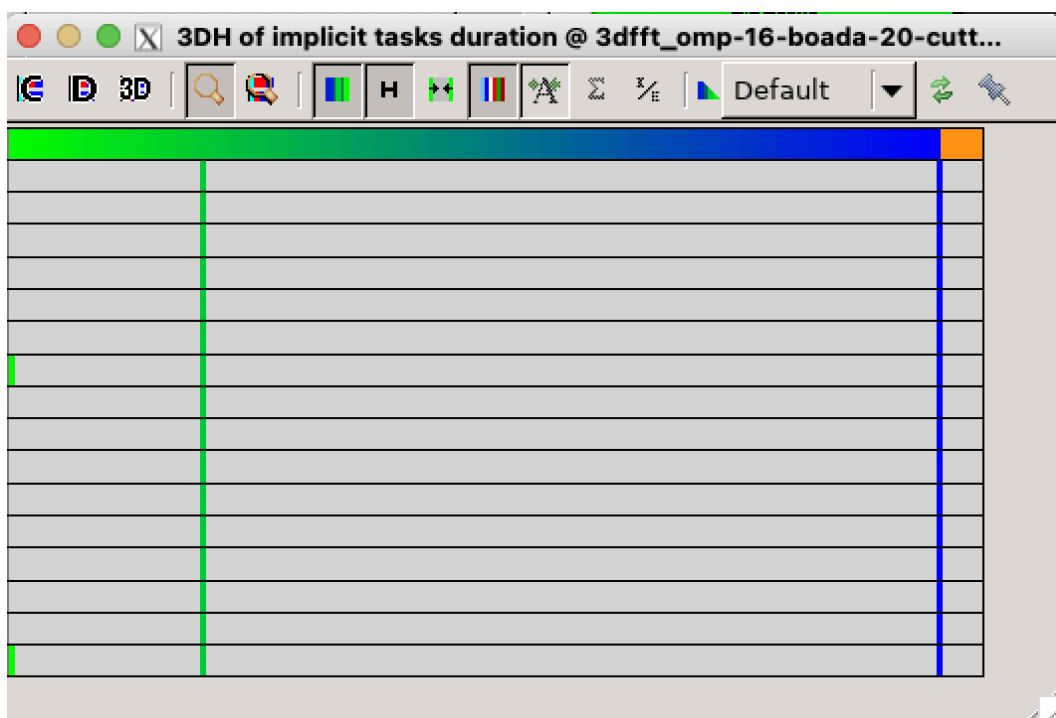
Punto 9

1.2.6. Paraver hints explicit/implicit tasks

1. Implicit tasks



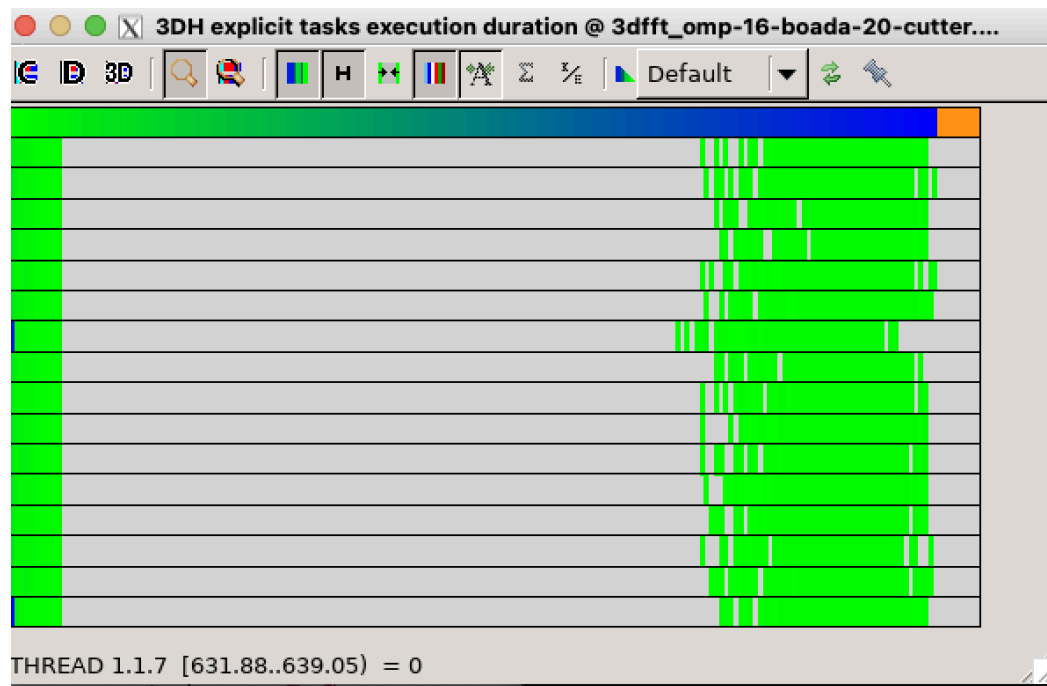
La visualización de **Implicit tasks duration** confirma que en cada región paralela solo se generan tantas tareas implícitas como hilos (16 en nuestro caso), cada una de duración similar a la región completa. Estas tareas son largas y estables, por lo que no representan un coste significativo. El problema de eficiencia proviene de las tareas explícitas creadas por los taskloop, que son extremadamente numerosas y de granularidad muy fina, tal y como se verá en la vista de explicit tasks.



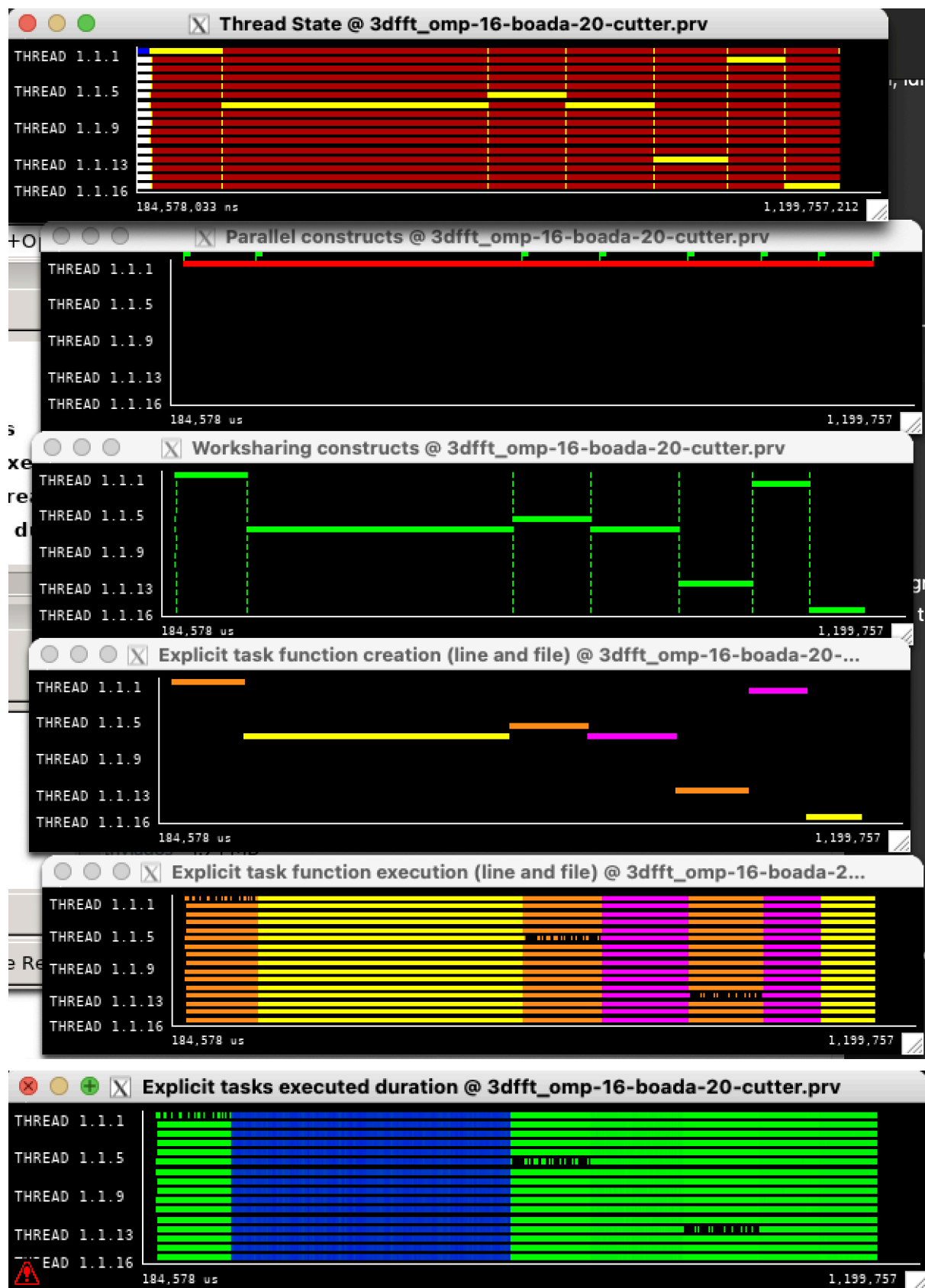
2. Explicit tasks



La vista de **Explicit tasks duration** muestra claramente el problema de granularidad fina: cada hilo ejecuta un gran número de tareas muy cortas, de apenas unas decenas de microsegundos.



1.2.6. Paraver configuration explicit/implicit tasks



1.3 Parallel improvements and analysis of 3DFFT: reducing the parallelisation overheads

Overview of whole program execution metrics						
Number of processors	1	2	4	8	16	32
Elapsed time (sec)	1.19	0.79	0.56	0.46	0.37	0.34
Speedup	1.00	1.51	2.13	2.61	3.22	3.52
Efficiency	1.00	0.75	0.53	0.33	0.20	0.11

Table 1: Analysis done on Thu Sep 25 04:25:30 PM CEST 2025, pacol114

Overview of the Efficiency metrics in parallel fraction, $\phi=83.02\%$						
Number of processors	1	2	4	8	16	32
Global efficiency	99.97%	83.92%	68.75%	48.11%	36.17%	22.37%
Parallelization strategy efficiency	99.97%	97.72%	95.98%	95.71%	96.73%	91.41%
Load balancing	100.00%	98.85%	97.37%	96.21%	98.36%	95.93%
In execution efficiency	99.97%	98.85%	98.57%	99.47%	98.34%	95.29%
Scalability for computation tasks	100.00%	85.88%	71.63%	50.27%	37.39%	24.47%
IPC scalability	100.00%	86.01%	71.45%	49.82%	37.35%	24.64%
Instruction scalability	100.00%	100.00%	99.99%	99.98%	99.96%	99.93%
Frequency scalability	100.00%	99.85%	100.27%	100.92%	100.15%	99.38%

Table 2: Analysis done on Thu Sep 25 04:25:30 PM CEST 2025, pacol114

Statistics about explicit tasks in parallel fraction						
Number of processors	1	2	4	8	16	32
Number of explicit tasks executed (total)	70.0	140.0	280.0	560.0	1120.0	1792.0
LB (number of explicit tasks executed)	1.0	0.99	0.93	0.89	0.83	0.86
LB (time executing explicit tasks)	1.0	0.99	0.98	0.97	0.99	0.97
Time per explicit task (average us)	14134.54	8225.91	4931.36	3511.44	2355.53	2235.59
Overhead per explicit task (synch %)	0.01	2.28	4.05	4.26	2.71	8.19
Overhead per explicit task (sched %)	0.02	0.02	0.03	0.03	0.04	0.05
Number of taskwait/taskgroup (total)	7.0	7.0	7.0	7.0	7.0	7.0

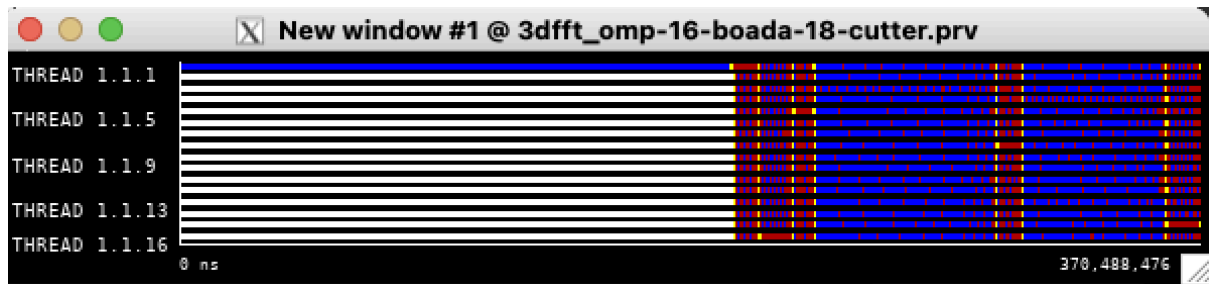
Table 3: Analysis done on Thu Sep 25 04:25:30 PM CEST 2025, pacol114

Tras aplicar la Optimización 1 (mover el taskloop al bucle externo), los resultados de Model factors muestran una mejora significativa. En la Tabla 1, el speedup pasa de $S_{16}=1.07$ en la versión inicial a $S_{16}=3.22$ en la optimizada, eliminando el slowdown observado previamente.

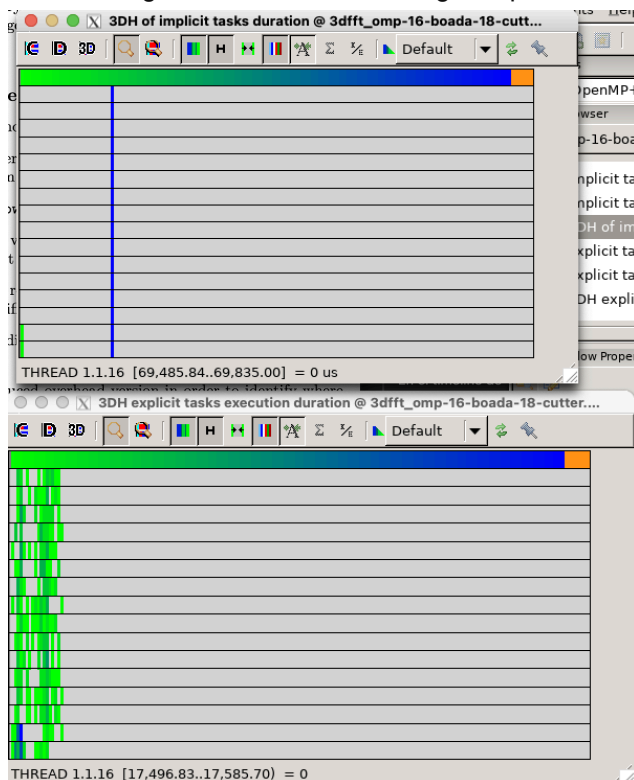
En la Tabla 2 se observa que la *in-execution efficiency* se mantiene cercana al 100 % incluso a 32 hilos, mientras que en la versión inicial caía por debajo del 50 %. La *scalability for computation tasks* también mejora notablemente (37 % a 16 hilos frente al 10 % anterior).

En la Tabla 3, el número de tareas explícitas se reduce drásticamente (de cientos de miles a apenas unas miles), aumentando la duración media de cada tarea a varios milisegundos y reduciendo el overhead de sincronización del 52 % al 2.7 % a 16 hilos.

En conclusión, la optimización resolvió el problema de granularidad fina: la escalabilidad ahora está limitada únicamente por la fracción paralela ($\phi \approx 83 \%$), coherente con la predicción de la ley de Amdahl ($S^\infty \approx 6$).



En el timeline de 16 hilos de la versión optimizada se observa un cambio importante respecto a la versión inicial. Mientras que antes predominaban tareas muy cortas y fragmentadas, ahora aparecen bloques de ejecución azul mucho más largos, indicando tareas explícitas de mayor duración y mejor aprovechamiento de los hilos. El porcentaje de tiempo en sincronización (rojo) y en scheduling (amarillo) se reduce visiblemente, lo que concuerda con la caída de overhead mostrada en la Tabla 3 de Modelfactors. Esta comparación visual confirma que la Optimización 1 ha conseguido reducir la sobrecarga de paralelización mediante una granularidad más gruesa.



En la versión optimizada, el histograma de tareas explícitas muestra un cambio drástico respecto a la inicial: las duraciones pasan de decenas de microsegundos a varios milisegundos, con un número total de tareas muy inferior. Esto confirma que se ha conseguido engordar la granularidad, reduciendo la sobrecarga de creación y sincronización.

Por el contrario, el histograma de tareas implícitas se mantiene invariable, con 16 tareas largas y estables, lo que confirma que nunca fueron un factor limitante.

La comparación con la versión inicial corrobora lo observado en Modelfactors: la optimización redujo los overheads y mejoró la escalabilidad global.

¿Crees que usar una granularidad muy fina es siempre la mejor estrategia de paralelización? ¿Por qué?

No, una granularidad muy fina no siempre es la mejor estrategia. En la versión inicial, la granularidad excesivamente fina generó cientos de miles de tareas explícitas con duraciones de apenas 30–50 μ s, provocando un overhead de sincronización superior al 50 % (Tabla 3). Esto llevó a que el programa dejara de escalar e incluso empeorara con más de 8 hilos.

Tras engordar la granularidad, el número de tareas se redujo drásticamente y la duración media subió a varios milisegundos, con overhead de sincronización <10 %. La eficiencia de ejecución mejoró notablemente y el programa escaló hasta 16–32 hilos.

Por tanto, la estrategia óptima es encontrar un **equilibrio**: tareas demasiado pequeñas introducen mucho overhead, mientras que tareas demasiado grandes pueden producir desbalance de carga.

¿El overhead de sincronización fue constante o dependía del número de hilos?

El overhead de sincronización fue claramente una **función del número de hilos**. En la versión inicial aumentó de forma dramática al crecer p: 11,7 % (8 hilos), 52 % (16 hilos) y 149 % (32 hilos).

Esto indica que cuantos más hilos había compitiendo por tareas pequeñas, mayor era el coste de sincronización y el tiempo perdido en esperas.

En la versión optimizada, el overhead de sincronización se mantuvo mucho más bajo y creció lentamente con p (2,7 % a 16 hilos, 8,2 % a 32 hilos), lo que confirma que reducir la granularidad fina estabilizó este coste.

¿Cómo resolviste el problema de overhead de la estrategia inicial de granularidad fina?

El problema se resolvió **coarsening la granularidad**:

- Se **comentó el taskloop del bucle interno** y se **activó el taskloop del bucle externo** en cada región paralela.
- Esto redujo drásticamente el número de tareas explícitas (de cientos de miles a unos miles), aumentando su duración media de microsegundos a milisegundos.
- Como resultado, el overhead de sincronización y planificación cayó más de 10 veces, la eficiencia de ejecución subió cerca del 100 % y el programa escaló hasta 16–32 hilos.

1.4 Parallel improvements and analysis of 3DFFT: improving ϕ

Overview of whole program execution metrics						
Number of processors	1	2	4	8	16	32
Elapsed time (sec)	1.19	0.67	0.39	0.25	0.17	0.12
Speedup	1.00	1.77	3.09	4.69	7.18	9.91
Efficiency	1.00	0.89	0.77	0.59	0.45	0.31

Table 1: Analysis done on Thu Sep 25 05:47:58 PM CEST 2025, paco1114

Overview of the Efficiency metrics in parallel fraction, $\phi=99.42\%$						
Number of processors	1	2	4	8	16	32
Global efficiency	99.97%	88.27%	76.84%	58.44%	44.70%	31.00%
Parallelization strategy efficiency	99.97%	98.43%	97.00%	96.54%	93.98%	92.30%
Load balancing	100.00%	98.85%	98.24%	97.63%	96.96%	95.84%
In execution efficiency	99.97%	99.58%	98.74%	98.88%	96.92%	96.31%
Scalability for computation tasks	100.00%	89.68%	79.22%	60.53%	47.57%	33.58%
IPC scalability	100.00%	90.42%	79.24%	59.92%	47.62%	34.30%
Instruction scalability	100.00%	100.00%	99.99%	99.98%	99.96%	99.93%
Frequency scalability	100.00%	99.18%	99.97%	101.04%	99.94%	97.98%

Table 2: Analysis done on Thu Sep 25 05:47:58 PM CEST 2025, paco1114

Statistics about explicit tasks in parallel fraction						
Number of processors	1	2	4	8	16	32
Number of explicit tasks executed (total)	80.0	160.0	320.0	640.0	1280.0	2048.0
LB (number of explicit tasks executed)	1.0	1.0	0.96	0.89	0.81	0.9
LB (time executing explicit tasks)	1.0	1.0	0.99	0.99	0.98	0.97
Time per explicit task (average us)	14777.96	8237.06	4662.09	3048.8	1935.58	1698.16
Overhead per explicit task (synch %)	0.0	1.54	2.99	3.3	5.76	7.07
Overhead per explicit task (sched %)	0.02	0.02	0.02	0.03	0.05	0.07
Number of taskwait/taskgroup (total)	8.0	8.0	8.0	8.0	8.0	8.0

Table 3: Analysis done on Thu Sep 25 05:47:58 PM CEST 2025, paco1114

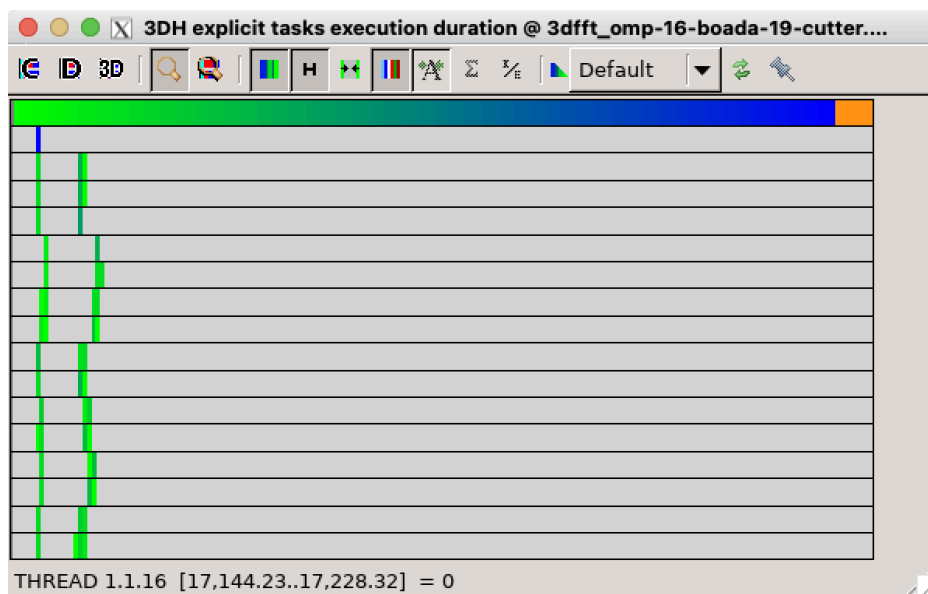
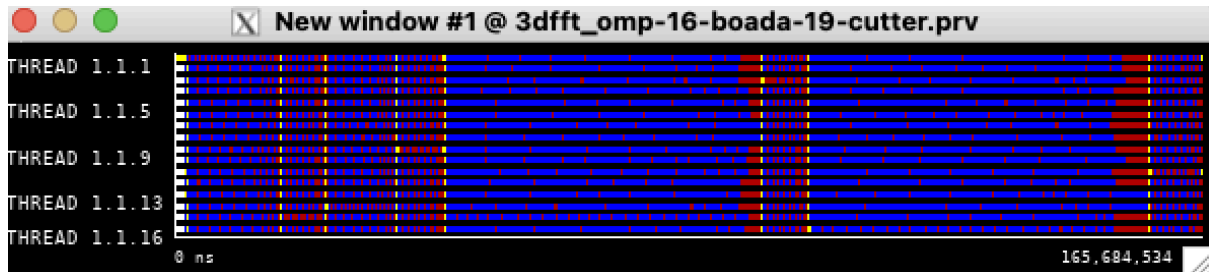
Tras resolver el problema de granularidad fina, el rendimiento seguía limitado por la fracción paralela del programa ($\phi \approx 83\%$), lo que restringía el speedup máximo teórico a ≈ 6 según la ley de Amdahl. Al paralelizar la función `init_complex_grid`, la fracción paralela aumentó hasta 99.4% , elevando el límite teórico a más de 160. En consecuencia, el speedup medido alcanzó valores de $S_{16}=7.18$ y $S_{32}=9.91$, con una eficiencia aún razonable.

Comparando las tres versiones:

- Inicial: $\phi \approx 83\%$, overhead altísimo, sin escalabilidad más allá de 4 hilos.
- Reduced overhead: $\phi \approx 83\%$, overhead bajo, escalabilidad hasta 16–32 hilos pero limitada por Amdahl.

- Improved ϕ : $\phi \approx 99.4\%$, escalabilidad notable hasta 32 hilos, limitado ahora por la arquitectura.

Esto demuestra la importancia de combinar estrategias: primero eliminar overheads, y después aumentar la fracción paralela.



El diagrama de **Instantaneous Parallelism** de la versión final muestra largos intervalos con paralelismo elevado y estable, cercanos al número máximo de hilos disponibles (16). Las caídas puntuales corresponden a fases de sincronización o finalización de regiones paralelas. En comparación con la versión inicial, donde el paralelismo efectivo era muy bajo y fragmentado debido a la granularidad fina, y con la versión reducida en overhead, donde aún persistían fases secuenciales, la versión con `init_complex_grid` paralelizada logra mantener un paralelismo mucho mayor durante casi toda la ejecución. Esto confirma visualmente el incremento de la fracción paralela (ϕ) y la mejora en la escalabilidad.

Once the overhead problem was solved, what was limiting the performance of the parallel strategy?

Una vez eliminados los overheads de sincronización y planificación (Optimización 1), el rendimiento ya no estaba limitado por la granularidad de las tareas, sino por la **fracción paralela ϕ del programa**. En la versión reducida en overhead, $\phi \approx 83\%$, lo que según la ley de Amdahl limita el speedup máximo teórico a ≈ 6 , independientemente del nº de hilos.

Al paralelizar `init_complex_grid` en la tercera versión, se aumentó ϕ hasta $\approx 99.4\%$. A partir de ahí, el límite del rendimiento pasó a ser la **eficiencia de escalabilidad computacional** (IPC, frecuencia, recursos de hardware compartidos), ya que teóricamente el speedup podría crecer mucho más, pero en la práctica el procesador no puede mantener eficiencia perfecta a gran nº de hilos.

Compare the parallel fraction of the three versions: initial, reduced overhead, and improved parallel fraction.

- **Versión inicial (granularidad fina):** $\phi \approx 83\%$. La baja escalabilidad se debía a overheads altísimos (sincronización hasta 149 % a 32 hilos).
- **Versión reducida en overhead:** $\phi \approx 83\%$ (igual que la inicial, porque no se paralelizó más código). La mejora vino de engordar la granularidad, reduciendo el overhead de sincronización a $<10\%$ y consiguiendo speedup ≈ 3.2 (16 hilos).
- **Versión mejorada en ϕ (con `init_complex_grid` paralelizado):** $\phi \approx 99.4\%$. Esto permitió un speedup mucho mayor ($S_{16} \approx 7.2$, $S_{32} \approx 9.9$) y que el límite del rendimiento pasara a ser el hardware, no el modelo de programación.